



Assignment Report

Hypersonics and Advanced Propulsion

Submitted By:

Toran Bahadur Niroula
078BAS043

Submitted To:

Asst. Prof. Dr. Sudip Bhattarai
Department of Mechanical and Aerospace Engineering
Pulchowk Campus, IOE, Tribhuvan University

April, 2026

Contents

Hypersonics	1
1 Introduction	1
2 Governing Equations and Methods	1
2.1 Local Inclination Methods	1
2.2 Shock Polar	1
3 CFD Setup	2
4 Cylinder Results	3
4.1 Pressure Distribution	3
4.2 CFD Results	4
5 Shock-Shock Interaction Results	6
5.1 Polar Classification	6
5.2 Type I CFD Results	7
5.3 Type II CFD Results	9
6 Discussion	11
 Advanced Propulsion	 12
1 Introduction	12
2 Intake Design	12
2.1 Design Requirements	12
2.2 Oblique Shock Design Procedure	12
3 CFD Setup	13
4 Inviscid CFD Results	14
4.1 Performance Comparison	15
5 RANS CFD Results	16
6 Discussion	16
Appendix: Python Code	17

Hypersonics

1. Introduction

This part addresses two problems at a common freestream condition of $M_\infty = 3$ to 6 at ISA 20 km:

- (i) pressure distribution on a circular cylinder at $M_\infty = 6$, computed via three local inclination methods and validated against inviscid CFD
- (ii) shock polar classification and CFD confirmation of regular (Type I) and Mach interaction (Type II) shock-shock interactions at $M_\infty = 3$.

Table 1: Freestream conditions (ISA 20 km, $\gamma = 1.4$, $C_p = 1004.5 \text{ J/(kg K)}$).

Quantity	Symbol	Value	Unit
Static pressure	p_∞	5529	Pa
Density	ρ_∞	0.088	kg/m ³
Temperature	T_∞	216.65	K
Speed of sound	a_∞	≈ 295	m/s
Dynamic viscosity	μ	$\approx 1.42 \times 10^{-5}$	Pa·s

2. Governing Equations and Methods

2.1. Local Inclination Methods

At hypersonic Mach numbers, each surface element is treated as an independent wedge. On the leeward side all three methods set $C_p = 0$.

Modified Newtonian Theory applies the relation:

$$C_p = C_{p,\max} \sin^2 \theta, \quad C_{p,\max} = \frac{2}{\gamma M_\infty^2} \left[\frac{p_{02}}{p_\infty} - 1 \right], \quad (1)$$

where p_{02}/p_∞ follows from the Rayleigh pitot formula. At $M_\infty = 6$, $C_{p,\max} \approx 1.831$.

Tangent Wedge Method applies oblique shock relations for the local half-angle θ , solving the θ - β - M relation for β and hence p/p_∞ . Beyond the sonic point, Modified Newtonian Theory is used as a fallback.

Shock Expansion Theory starts from an oblique shock at the stagnation point and marches downstream via Prandtl-Meyer expansion:

$$\nu(M_2) = \nu(M_1) + |\Delta\theta|, \quad \nu(M) = \sqrt{\frac{\gamma+1}{\gamma-1}} \tan^{-1} \sqrt{\frac{\gamma-1}{\gamma+1} (M^2 - 1)} - \tan^{-1} \sqrt{M^2 - 1}. \quad (2)$$

2.2. Shock Polar

A shock polar traces all oblique shock solutions in the $(\theta, p/p_\infty)$ plane:

$$\frac{p_2}{p_1} = 1 + \frac{2\gamma}{\gamma+1} (M_1^2 \sin^2 \beta - 1), \quad \tan \theta = \frac{2 \cot \beta (M_1^2 \sin^2 \beta - 1)}{M_1^2 (\gamma + \cos 2\beta) + 2}. \quad (3)$$

A **Type I** (regular) interaction occurs when the two downstream polars intersect on the weak-shock branch. A **Type II** (Mach interaction) occurs when no intersection exists and a Mach stem forms.

3. CFD Setup

All simulations use `rhoCentralFoam` with Kurganov-Tadmor flux, `vanLeer` reconstruction, $Co_{\max} = 0.4$, and slip walls.

Table 2: CFD case summary.

Case	M_{∞}	Cells	Solver	Notes
Cylinder	6	40,000	<code>rhoCentralFoam</code>	
Type I ramps	3	33,000	<code>rhoCentralFoam</code>	$\theta = \pm 15^\circ$, Kantrowitz ratio: 1.147
Type II ramps	3	40,500	<code>rhoCentralFoam</code>	$\theta = \pm 30^\circ$, Kantrowitz ratio: 1.38

The cylinder domain is pear-shaped to align mesh lines with the bow shock. The `sonicFoam` solution was used to initialize the field to speed up simulation. For the shock-shock cases, the top-wall boundary was set to slip condition and adiabatic wall. Ramp of desired angles (15° and 30°) was constructed on the domain. Both Kantrowitz area ratios (1.147 and 1.38) fall within the limit of 1.39.

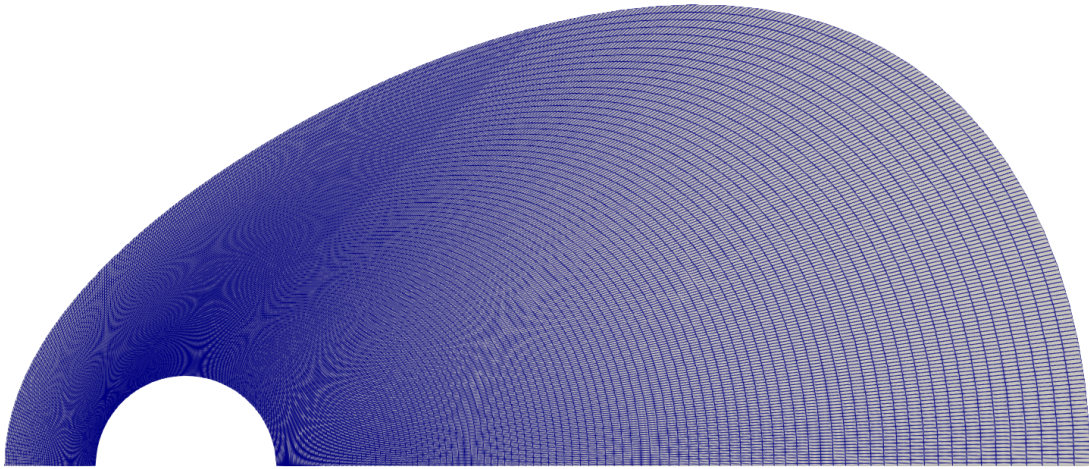


Figure 1: Cylinder mesh with approximately 40,000 cells and symmetry at centreline.

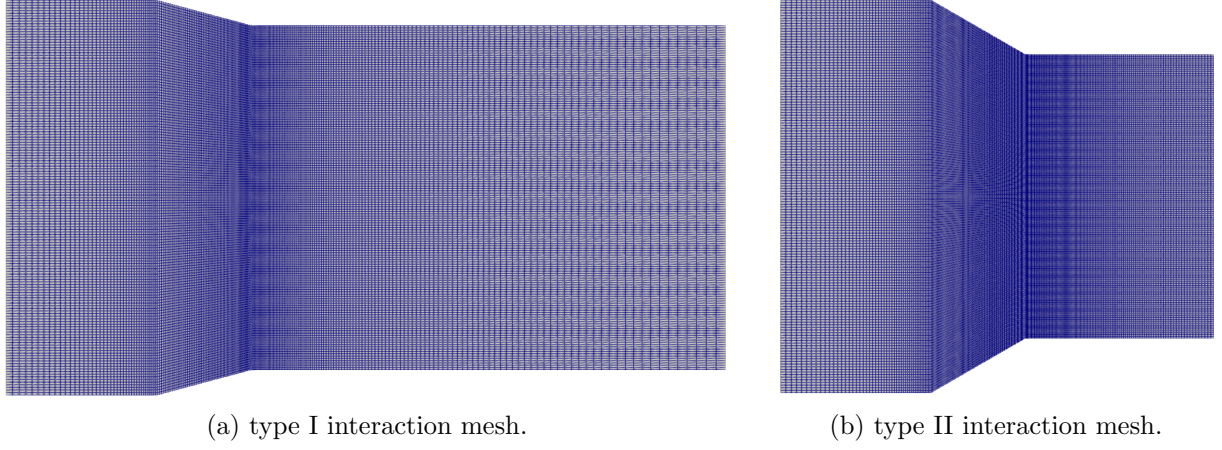


Figure 2: Cylinder total pressure and change in entropy.

4. Cylinder Results

4.1. Pressure Distribution

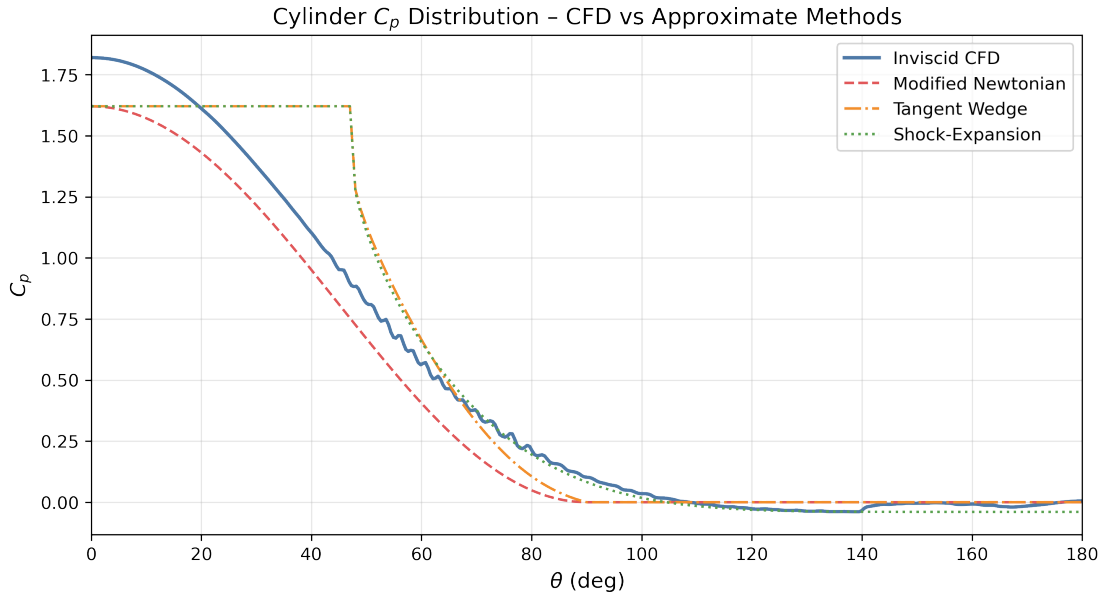


Figure 3: C_p distribution on the cylinder surface compared against inviscid CFD.

Modified Newtonian Theory tracks the C_p of inviscid CFD most uniformly across the windward face. The Tangent Wedge Method and Shock Expansion Theory perform better near the shoulder ($\theta \approx 40-70^\circ$) where the local geometry more closely resembles a wedge, but both methods struggle at the stagnation point where total-pressure-based C_p is applied up to the sonic point. The CFD peak $C_p \approx 1.8$ exceeds the Modified Newtonian prediction of ≈ 1.62 by roughly 11%. All inclination methods except Shock Expansion Theory collapse to $C_p = 0$ in the shadow region, which is consistent with the assumption of zero pressure perturbation on leeward surfaces. The leeward recirculation visible in Figure 4 is attributed to entropy layer vorticity, described by Crocco's Theorem as

$$\mathbf{V} \times \boldsymbol{\omega} = \nabla \mathcal{H}_0 - T \nabla S \quad (4)$$

where gradients in entropy S across streamlines passing through the curved bow shock drive the rotational flow observed in the wake.

4.2. CFD Results

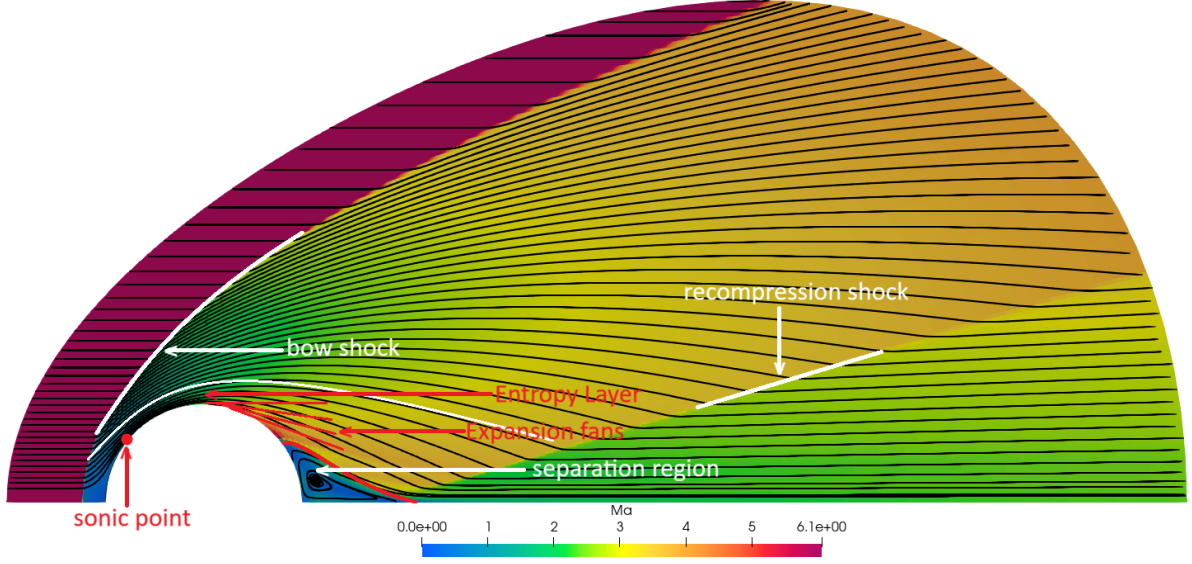


Figure 4: streamlines over the cylinder. Shows recirculation at leeward region possibly effect of entropy layer.



Figure 5: Cylinder flow field, $M_\infty = 6$.

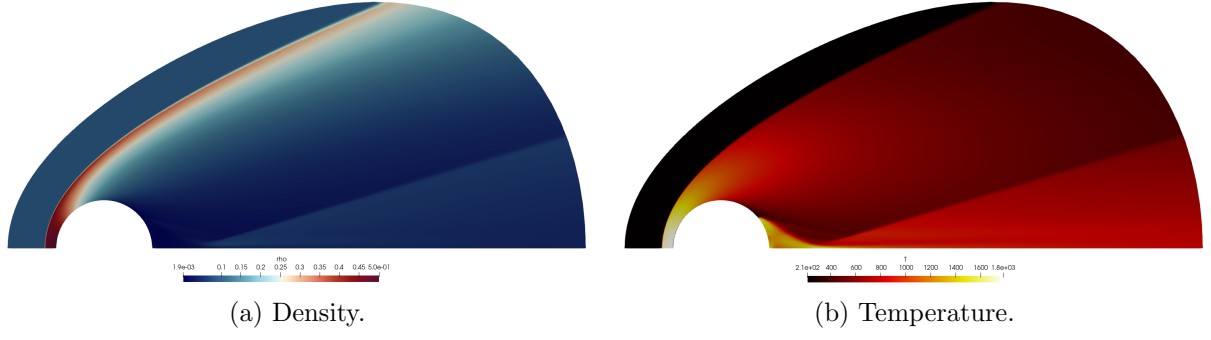


Figure 6: Cylinder density and temperature fields.

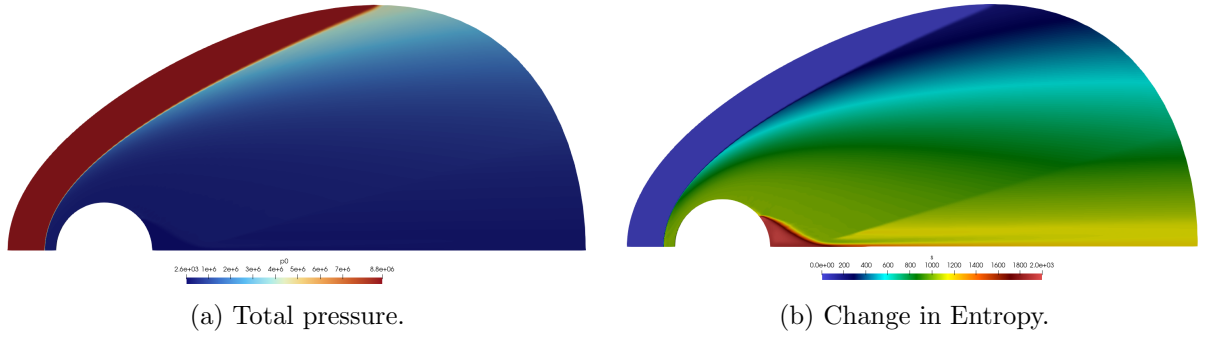


Figure 7: Cylinder total pressure and change in entropy.

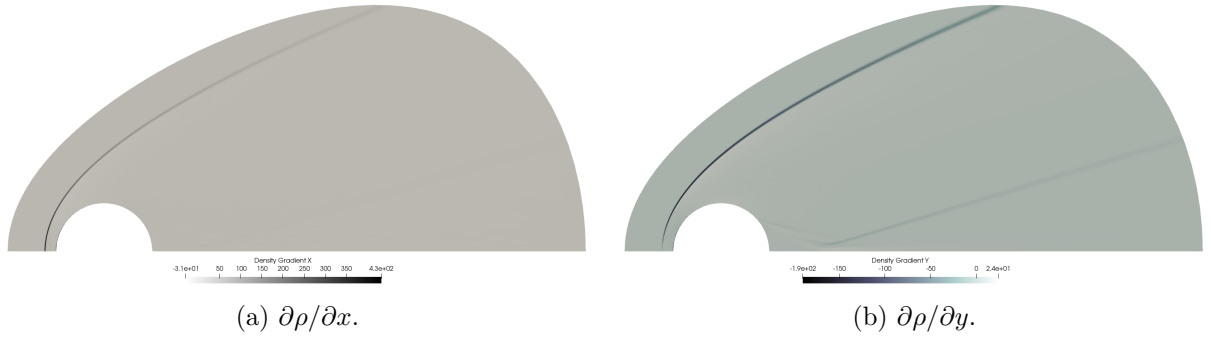
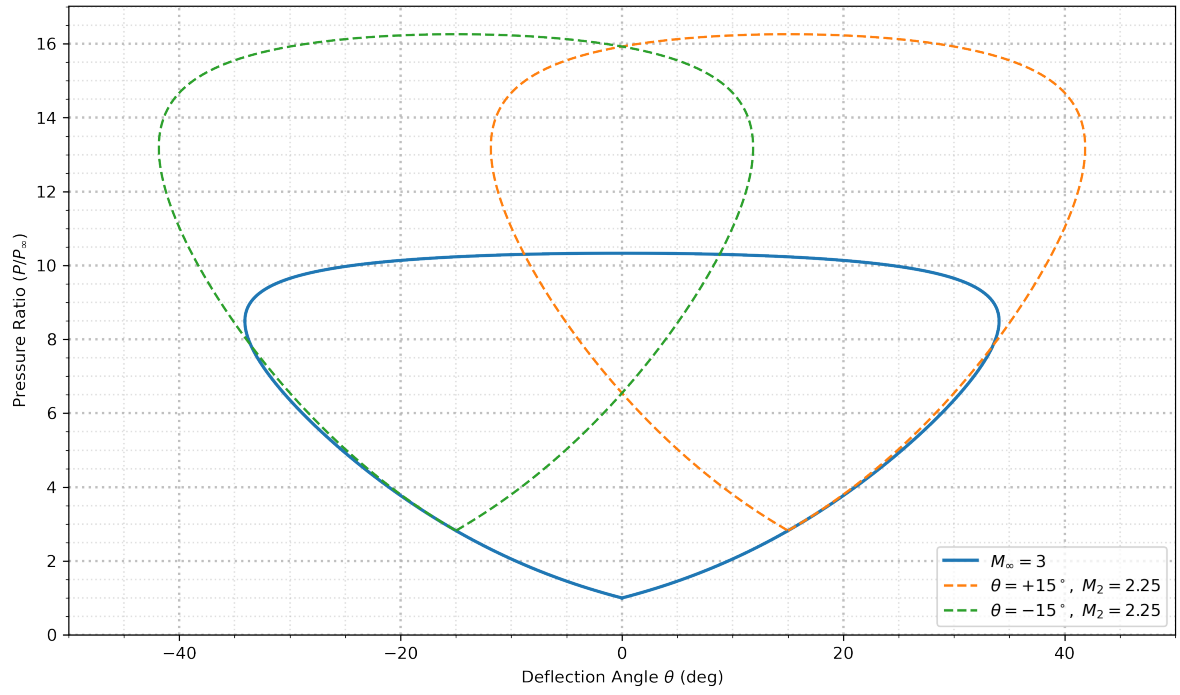


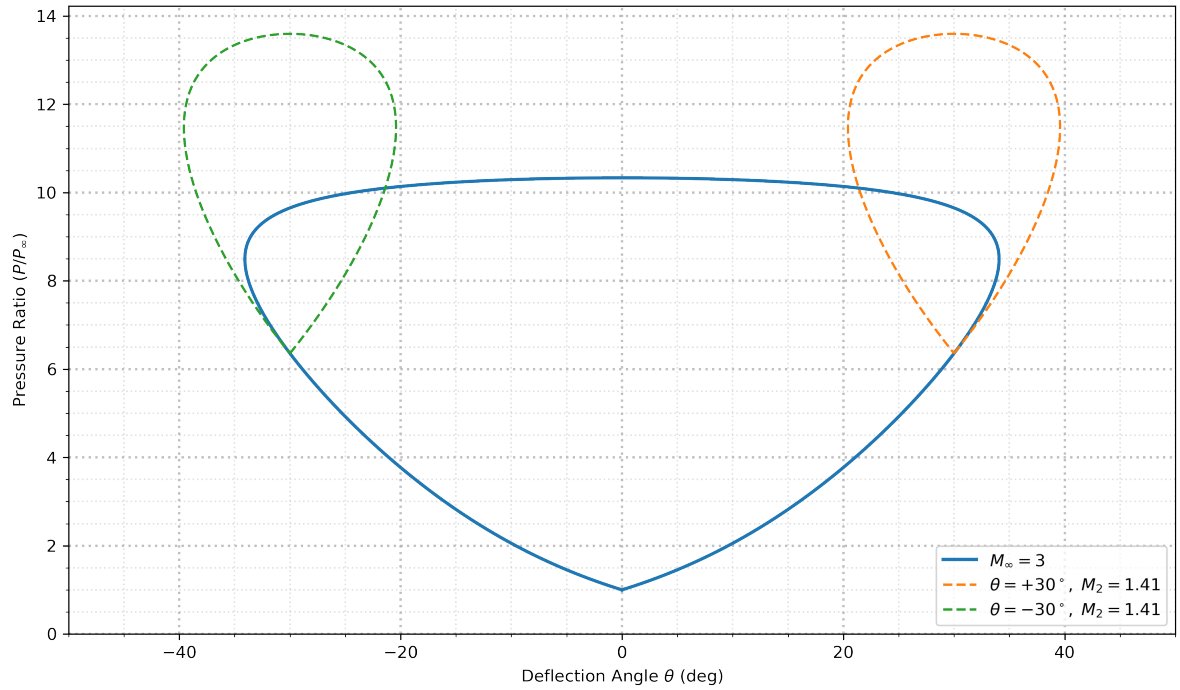
Figure 8: Numerical Schlieren for the cylinder.

5. Shock-Shock Interaction Results

5.1. Polar Classification



(a) Type I ($\theta = \pm 15^\circ$), regular interaction.



(b) Type II ($\theta = \pm 30^\circ$), Mach interaction.

Figure 9: Shock polars at $M_\infty = 3$.

5.2. Type I CFD Results

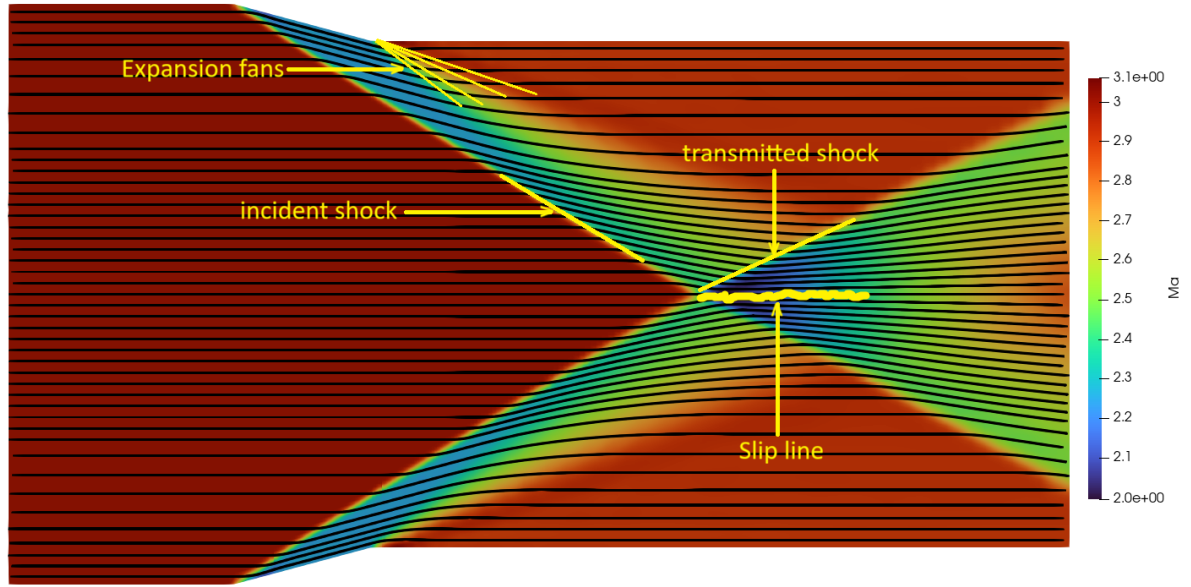


Figure 10: streamlines and flow features in regular interaction.

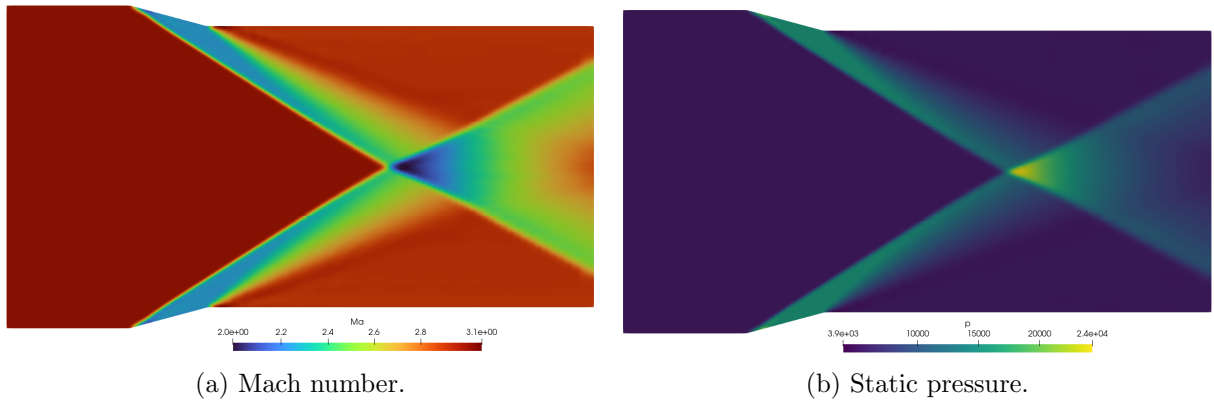


Figure 11: Type I flow field.

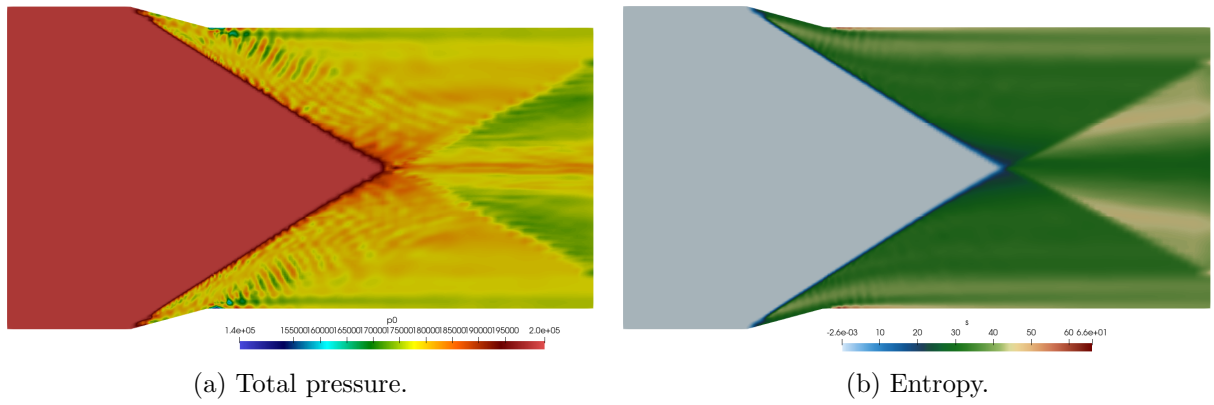


Figure 12: Type I total pressure and entropy.

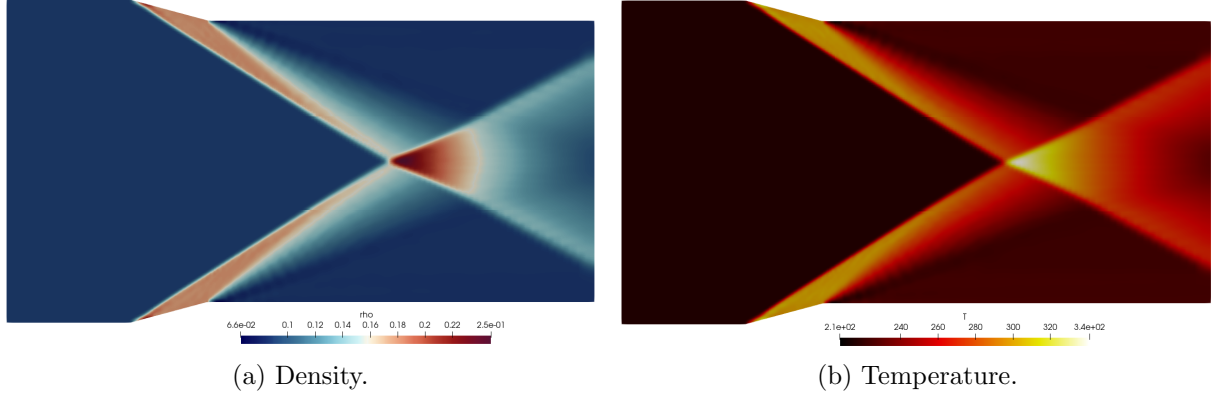


Figure 13: Type I density and temperature.

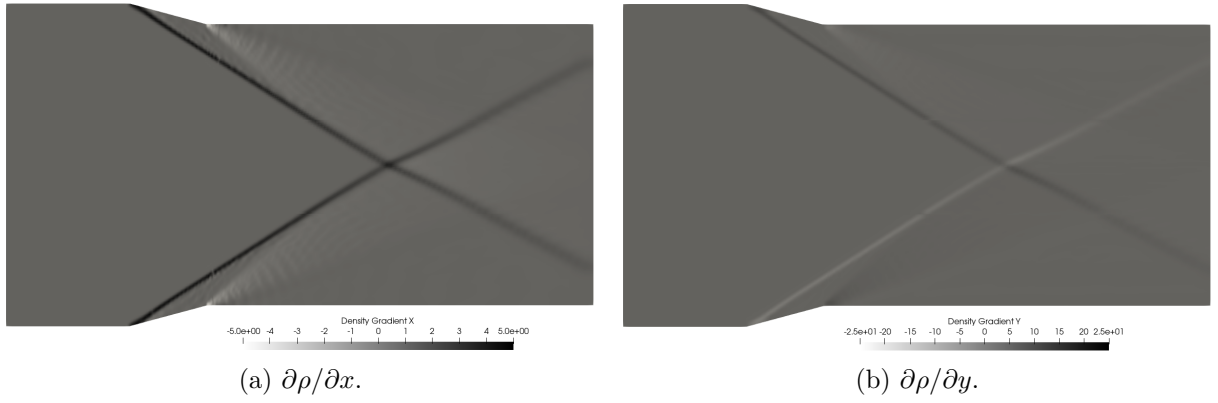


Figure 14: Numerical Schlieren for the Type I case.

The two incident shocks cross symmetrically and produce reflected shocks without a Mach stem. A narrow slip line is visible at the crossing point in the entropy contour (Figure 12b). The total pressure and entropy contours both show this slipline passing through the centreline.

5.3. Type II CFD Results

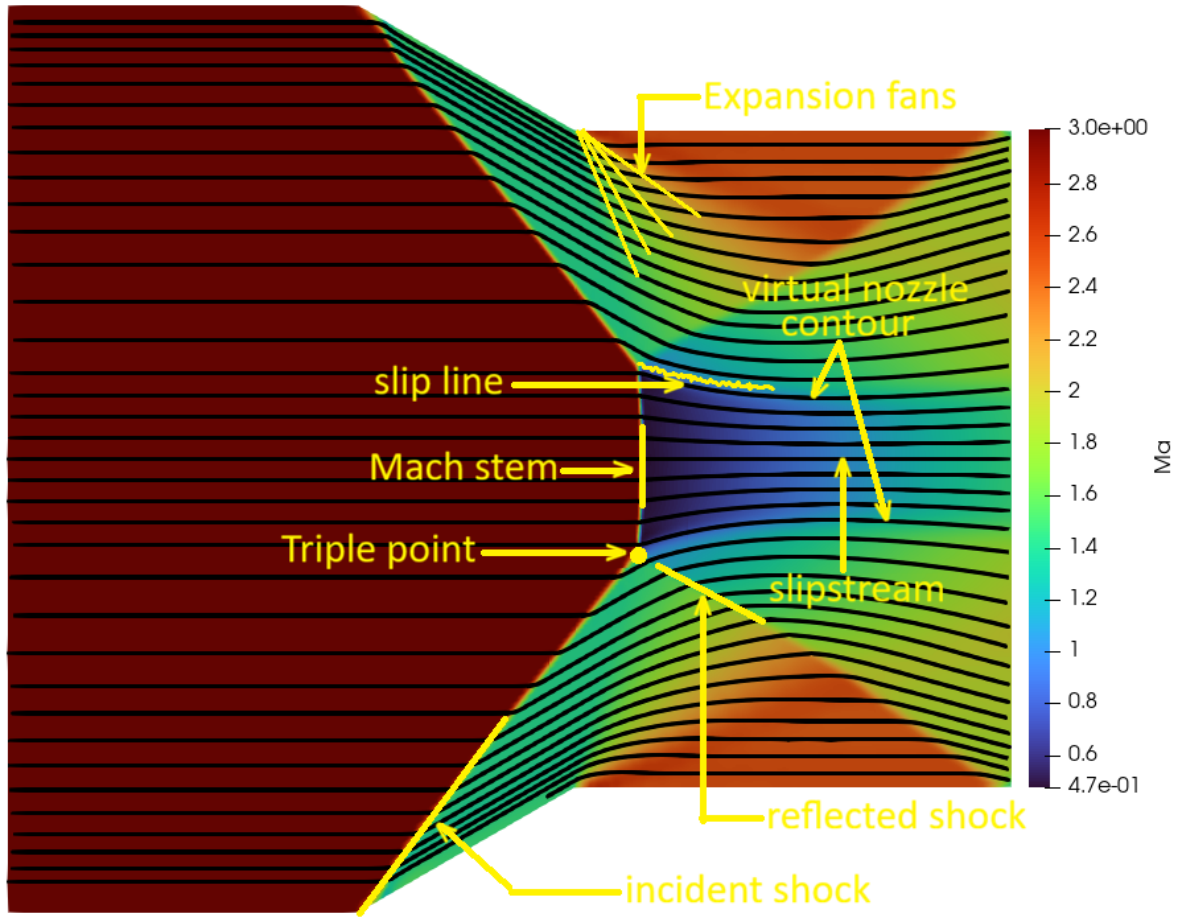


Figure 15: streamlines and flow features in Mach interaction.

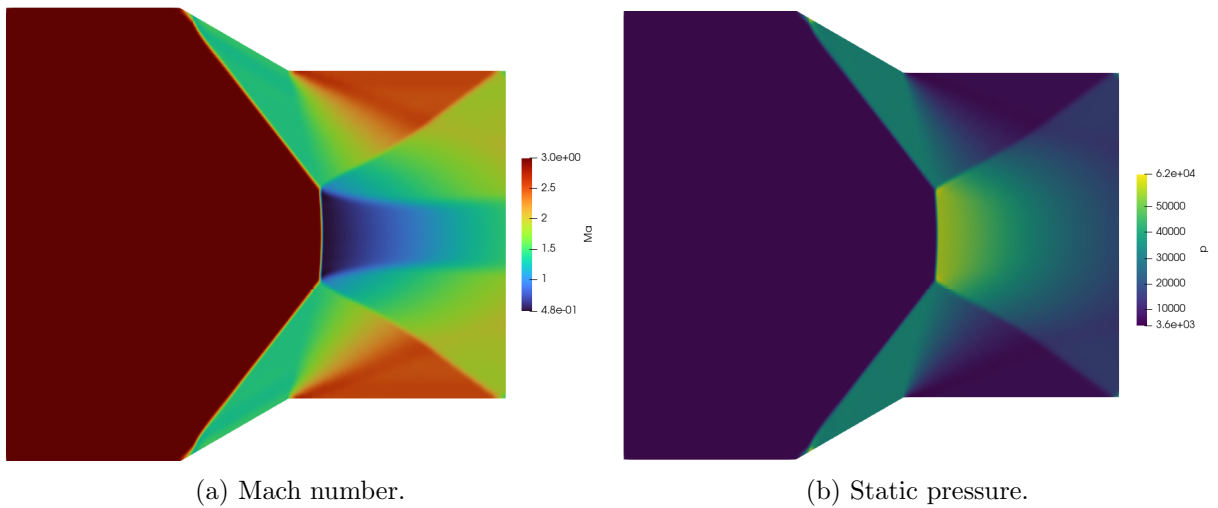


Figure 16: Type II flow field.

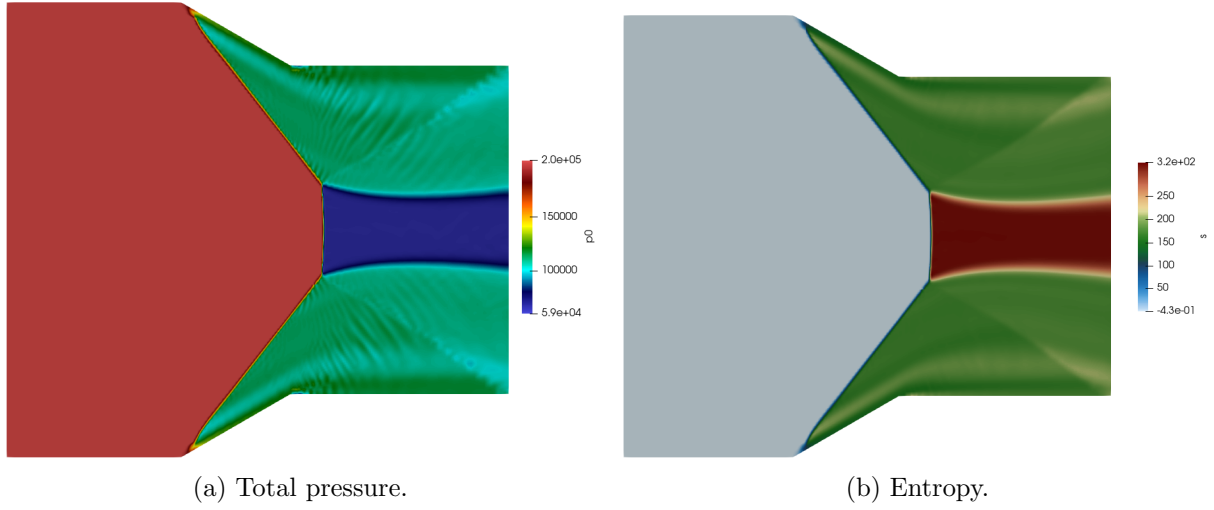


Figure 17: Type II total pressure and entropy.

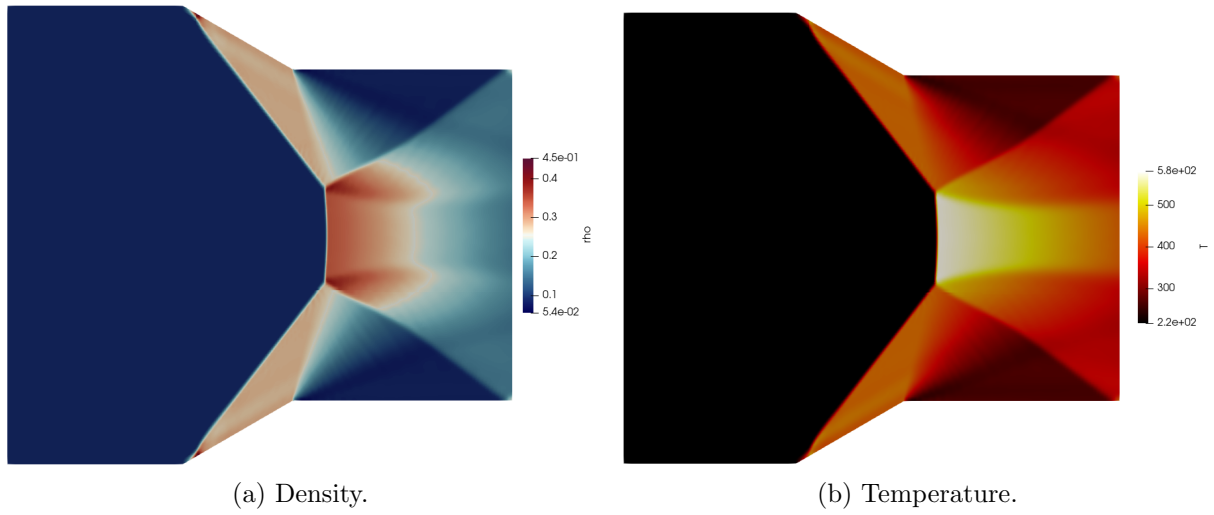


Figure 18: Type II density and temperature. The density jump across the Mach stem is substantially larger than across the oblique shocks.

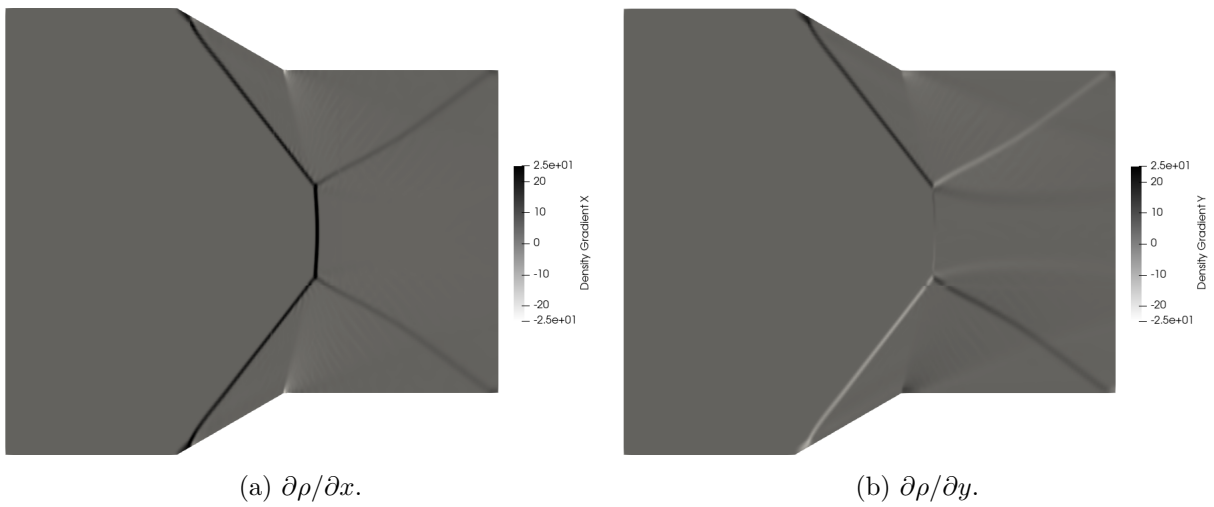


Figure 19: Numerical Schlieren for the Type II case.

A near-normal Mach stem spans the channel centreline, producing a subsonic pocket that is clearly visible in the Mach contour. Reflected shocks leave the stem and propagate downstream. A slipstream extends downstream from each triple point. The Mach stem generates a substantially larger entropy rise and total pressure loss than the oblique shocks. Expansion fans at the triple points then interact with the downstream flow, partially recovering the Mach number.

6. Discussion

Local inclination methods. Modified Newtonian Theory gives the best global agreement because its $\sin^2 \theta$ shape inherently captures the blunt-body pressure distribution. The Tangent Wedge Method and Shock Expansion Theory are more accurate near the shoulder ($\theta \approx 40\text{-}70^\circ$) but break down at the stagnation point where the wedge analogy fails. A stagnation point error of roughly 11% is observed.

Shock-shock interactions. CFD confirms the polar predictions at both deflection angles. The Type I case shows a clean regular reflection with negligible total pressure loss. The Type II case exhibits all features of a Mach interaction: a near-normal Mach stem, two triple points, a slipstream separating subsonic and supersonic regions, and expansion fans at the triple points. The Kantrowitz limit check confirms that the duct can start without a normal shock. Without satisfying this criterion, a standing normal shock at the ramp entrance would choke the duct.

Advanced Propulsion

1. Introduction

This part covers the oblique-shock based design and CFD validation of a single-ramp scramjet intake at $M_\infty = 6$ (ISA 20 km). An inviscid simulation confirms the design point performance targets, and a RANS simulation with $k-\omega$ SST quantifies viscous effects. Freestream conditions are as given in Table 1.

2. Intake Design

2.1. Design Requirements

Table 3: Scramjet intake design requirements.

Parameter	Symbol	Value
Flight Mach number	M_∞	6
Static pressure ratio	π	20
Isolator height	h_{iso}	50 mm

2.2. Oblique Shock Design Procedure

The intake is a single-ramp system. The ramp angle δ is found by simultaneously solving the oblique shock pressure ratio and the θ - β - M relation:

$$\frac{p_2}{p_1} = 1 + \frac{2\gamma}{\gamma + 1} (M_\infty^2 \sin^2 \beta - 1), \quad (5)$$

$$\tan \theta = \frac{2 \cot \beta (M_\infty^2 \sin^2 \beta - 1)}{M_\infty^2 (\gamma + \cos 2\beta) + 2}. \quad (6)$$

With $\pi = 20$, $M_\infty = 6$, $\gamma = 1.4$, the solution gives $\theta = 14.15^\circ$ and $\beta = 23.38^\circ$.

The total pressure recovery is:

$$\text{TPR} = \frac{p_{02}}{p_{01}} = \left[\frac{(\gamma + 1)M_\infty^2 \sin^2 \beta}{(\gamma - 1)M_\infty^2 \sin^2 \beta + 2} \right]^{\gamma/(\gamma-1)} \cdot \left[\frac{(\gamma + 1)}{2\gamma M_\infty^2 \sin^2 \beta - (\gamma - 1)} \right]^{1/(\gamma-1)}. \quad (7)$$

Ramp length and capture height follow from the shock impingement geometry:

$$L = \frac{h_{\text{iso}}}{\tan(\beta - \theta)}, \quad H = h_{\text{iso}} + L \tan \theta. \quad (8)$$

Table 4: Intake geometry from oblique shock design.

Parameter	Symbol	Value
Ramp angle	θ	14.15°
LE Shock angle	β_1	21.77°
Cowl Shock angle	β_2	26.15°
Ramp length	L	1.026 m
Capture height	H	0.301 m
Isolator height	h_{iso}	50 mm
Design TPR		0.50
Design π		20

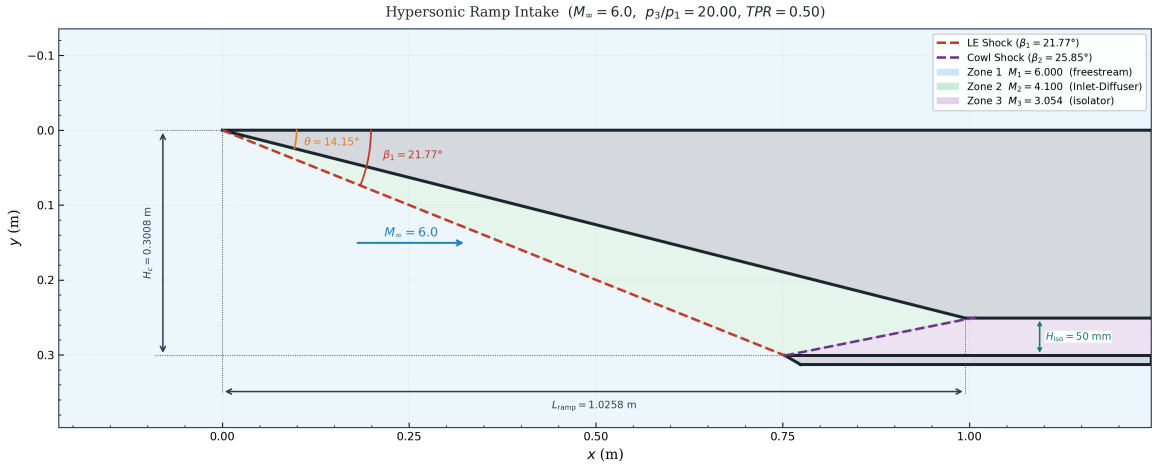


Figure 20: Scramjet ramp intake geometry.

3. CFD Setup

Both the inviscid and RANS simulations use `rhoCentralFoam` with Kurganov-Tadmor flux and `vanLeer` reconstruction. The RANS case adds the $k-\omega$ SST model. The structured mesh of 190,000 cells was generated with `blockMesh`. Refinement is applied near the ramp wall and in the oblique shock region.

Table 5: CFD simulation parameters.

Parameter	Inviscid	RANS
Solver	<code>rhoCentralFoam</code>	<code>rhoCentralFoam</code>
Turbulence	None	$k-\omega$ SST
Cell count	190,000	190,000
M_∞	6	6
Wall BC	Slip	No-slip, adiabatic
Flux scheme	Kurganov-Tadmor	
Δt	Adaptive ($\text{Co}_{\text{max}} = 0.4$)	

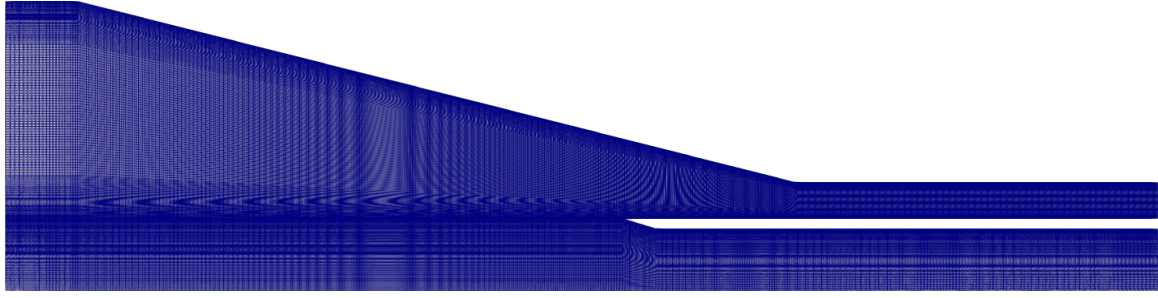


Figure 21: Computational mesh (190,000 cells).

4. Inviscid CFD Results

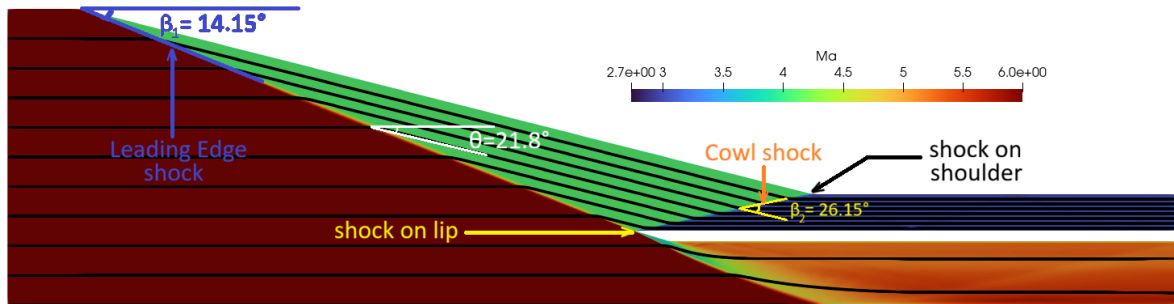


Figure 22: streamlines and flow features in single ramp intake.

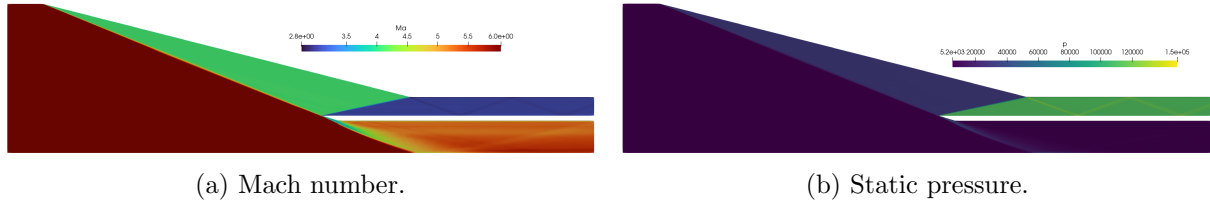


Figure 23: Inviscid flow field, $M_\infty = 6$.

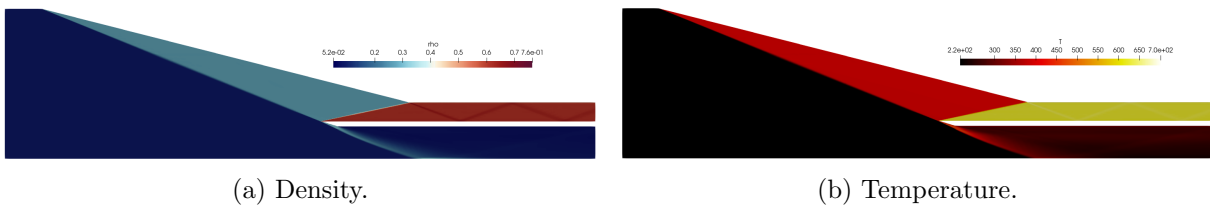


Figure 24: Inviscid density and temperature fields.

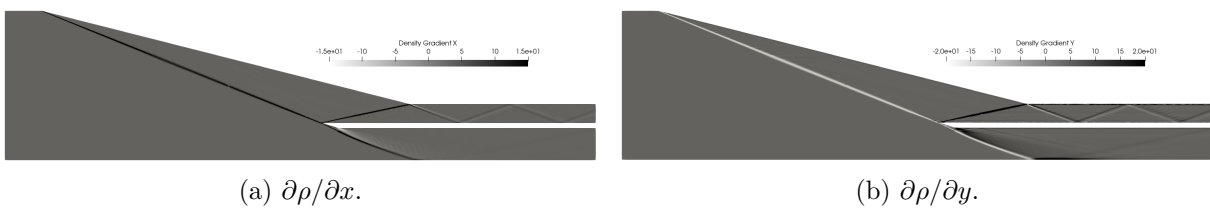


Figure 25: Numerical Schlieren for the inviscid intake.

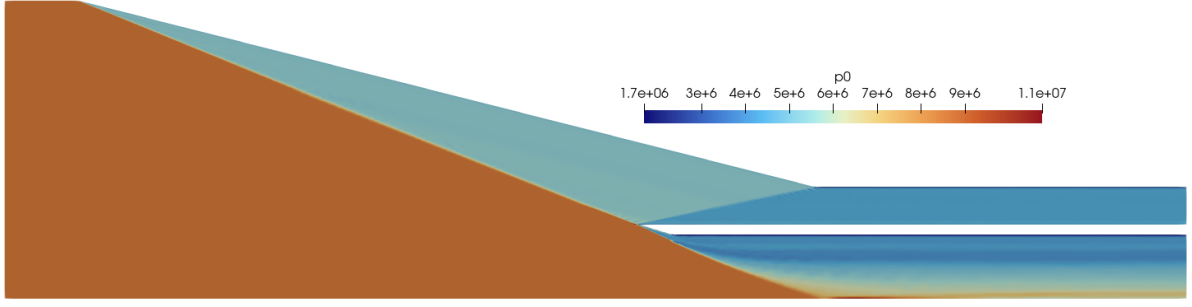


Figure 26: Total pressure field.

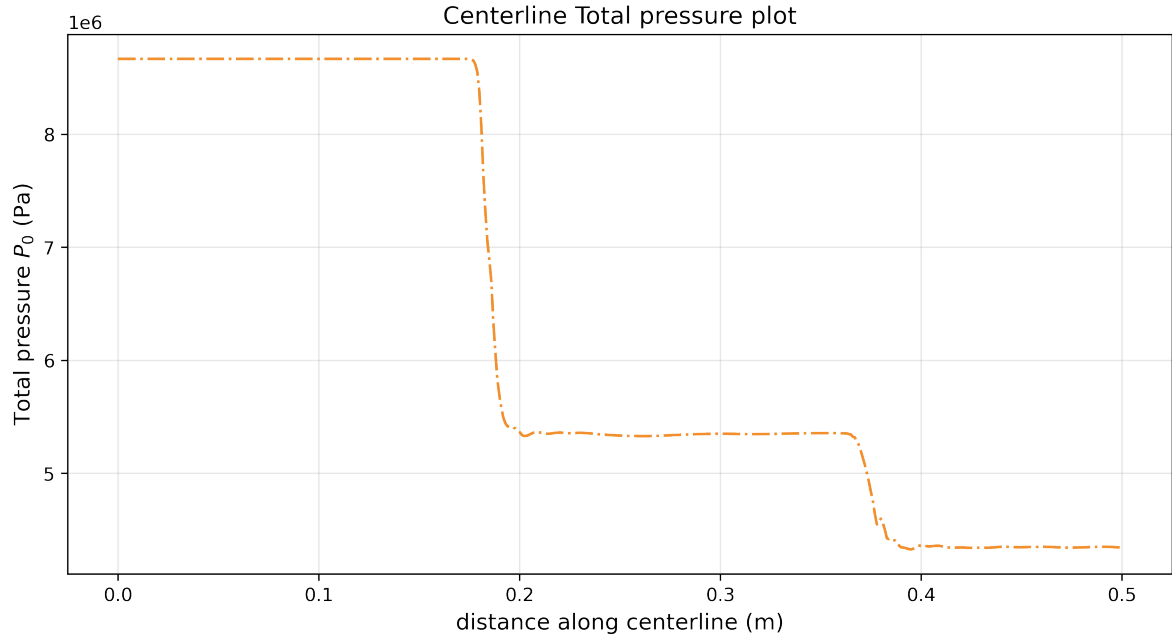


Figure 27: Centreline total pressure profile.

4.1. Performance Comparison

Table 6: Design vs. inviscid CFD performance at the isolator exit plane.

Parameter	Design	Inviscid CFD	Deviation
Total pressure recovery (TPR)	0.500	0.5013	+0.26%
Static pressure ratio π	20.00	19.928	-0.36%

Both parameters agree with design targets to within 0.4%. Since the inviscid CFD and design procedure share the same governing equations, the residual discrepancy reflects numerical dissipation in the Kurganov scheme and finite mesh resolution.

5. RANS CFD Results

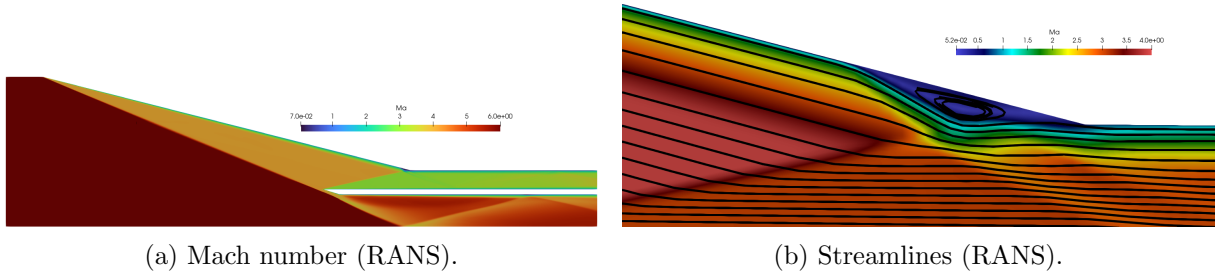


Figure 28: RANS flow field.

Boundary layer thickening on the ramp surface displaces the effective flow area relative to the inviscid case, and the shock impingement region becomes more diffuse. The streamlines show a thickened boundary layer near the shoulder where the cowl shock impinges, and a small separation bubble is visible at that location. Because of the boundary layer growth, the effective deflection angle seen by the flow increases, which means the leading-edge shock no longer impinges exactly at the cowl as in the inviscid case.

The $k-\omega$ SST model is well suited to this problem. Its blending between the $k-\omega$ formulation near the wall and $k-\varepsilon$ in the freestream handles the adverse pressure gradient at shock impingement. A known limitation is the fully turbulent assumption. If the boundary layer remains laminar over a significant portion of the ramp, the model will overestimate skin friction and underestimate any separation length.

6. Discussion

The less than 0.4% deviation between design and inviscid CFD confirms that the design procedure is internally consistent. Viscous effects (RANS) produce boundary-layer displacement that slightly reduces the effective compression area. The single-ramp TPR of 0.50 is characteristic of a strong oblique shock at Mach 6. A two or three ramp system would split the compression across weaker shocks and recover substantially more total pressure, at the cost of increased geometric complexity.

Appendix: Python Code

Listing 1: Pressure distribution via local inclination methods (cp_dist.py).

```
1 import numpy as np
2 import pandas as pd
3 from scipy.optimize import brentq
4
5 M = 6.0
6 gamma = 1.4
7
8 theta = np.linspace(0, np.pi, 181)
9 delta = np.pi / 2 - theta
10
11 def tbm(beta):
12     return np.arctan(2 / np.tan(beta)
13                     * (M**2 * np.sin(beta)**2 - 1)
14                     / (M**2 * (gamma + np.cos(2 * beta)) + 2))
15
16 betas = np.linspace(np.arcsin(1 / M) + 1e-6, np.pi / 2 - 1e-6, 1000)
17 tbm_vals = tbm(betas)
18 idx_max = np.argmax(tbm_vals)
19 delta_det = tbm_vals[idx_max]
20 beta_det = betas[idx_max]
21
22 def nu_pm(mach):
23     x = max(mach**2 - 1, 0.0)
24     return (np.sqrt((gamma + 1) / (gamma - 1))
25           * np.arctan(np.sqrt((gamma - 1) / (gamma + 1) * x))
26           - np.arctan(np.sqrt(x)))
27
28 Cp_stag = 2 / (gamma * M**2) * ((2 * gamma * M**2 - (gamma - 1)) / (gamma + 1) -
29                                1)
30
31 Cp_newt = np.where(delta >= 0, Cp_stag * np.sin(delta)**2, 0.0)
32
33 Mn1 = M * np.sin(beta_det)
34 M1n = np.sqrt(((gamma - 1) * Mn1**2 + 2) / (2 * gamma * Mn1**2 - (gamma - 1)))
35 M1 = M1n / np.sin(beta_det - delta_det)
36 p1 = (2 * gamma * Mn1**2 - (gamma - 1)) / (gamma + 1)
37 nu1 = nu_pm(M1)
38
39 nu_sh = nu1 + delta_det
40 M_sh = brentq(lambda mach: nu_pm(mach) - nu_sh, 1.0 + 1e-9, 50.0)
41 p_sh = p1 * ((1 + (gamma - 1) / 2 * M1**2)
42             / (1 + (gamma - 1) / 2 * M_sh**2)) ** (gamma / (gamma - 1))
43
44 Cp_tw = np.zeros_like(theta)
45 for i, d in enumerate(delta):
46     if d >= delta_det:
47         Cp_tw[i] = Cp_stag
48     elif d > 1e-6:
```

```

48     b = brentq(lambda b: tbm(b) - d, np.arcsin(1 / M) + 1e-9, beta_det - 1e
      -9)
49     Mn = M * np.sin(b)
50     Cp_tw[i] = 2 / (gamma * M**2) * ((2 * gamma * Mn**2 - (gamma - 1)) / (
      gamma + 1) - 1)
51
52 Cp_se = np.zeros_like(theta)
53 for i, d in enumerate(delta):
54     if d >= delta_det:
55         Cp_se[i] = Cp_stag
56
57     elif d > 1e-9:
58         nu2 = nu1 + (delta_det - d)
59         M2 = brentq(lambda mach: nu_pm(mach) - nu2, 1.0 + 1e-9, 50.0)
60         p2 = p1 * ((1 + (gamma - 1) / 2 * M1**2)
      / (1 + (gamma - 1) / 2 * M2**2)) ** (gamma / (gamma - 1))
61         Cp_se[i] = 2 / (gamma * M**2) * (p2 - 1)
62
63
64     elif abs(d) < 1e-9:
65         Cp_se[i] = 2 / (gamma * M**2) * (p_sh - 1)
66
67     else:
68         nu2 = nu_sh + abs(d)
69         nu2 = min(nu2, nu_pm(50.0) - 1e-9)
70         M2 = brentq(lambda mach: nu_pm(mach) - nu2, M_sh + 1e-9, 50.0)
71         p2 = p_sh * ((1 + (gamma - 1) / 2 * M_sh**2)
      / (1 + (gamma - 1) / 2 * M2**2)) ** (gamma / (gamma - 1))
72         Cp_se[i] = 2 / (gamma * M**2) * (p2 - 1)
73
74
75 df = pd.DataFrame({
76     "theta_deg":      np.degrees(theta),
77     "delta_deg":      np.degrees(delta),
78     "Cp_Newtonian":   Cp_newt,
79     "Cp_TangentWedge": Cp_tw,
80     "Cp_ShockExpansion": Cp_se,
81 })
82 df.to_csv("cylinder_Cp.csv", index=False)
83 print(df.to_string(index=False))

```

Listing 2: Shock polar generation (shockPolar.py).

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 GAMMA = 1.4
5 M_inf = 3
6 deflections = [15, -15]
7
8 def shock_polar(M, theta_offset=0.0, n_points=1000):
9     mu = np.degrees(np.arcsin(1 / M))
10    beta = np.linspace(mu, 90.0, n_points)
11    b_rad = np.radians(beta)
12    p_ratio = 1 + (2 * GAMMA / (GAMMA + 1)) * ((M * np.sin(b_rad))**2 - 1)

```

```

13     theta_rel = np.degrees(
14         np.arctan(
15             (2 / np.tan(b_rad)) *
16             ((M**2 * np.sin(b_rad)**2 - 1) /
17              (M**2 * (GAMMA + np.cos(2 * b_rad)) + 2))
18         )
19     )
20     theta_abs = np.concatenate([
21         theta_offset + theta_rel,
22         theta_offset - theta_rel[::-1]
23     ])
24     p_full = np.concatenate([p_ratio, p_ratio[::-1]])
25     return theta_abs, p_full
26
27 def weak_shock_angle(M, theta_deg):
28     mu = np.degrees(np.arcsin(1 / M))
29     betas = np.linspace(mu, 90.0, 10000)
30     for b in betas:
31         b_rad = np.radians(b)
32         t = np.degrees(
33             np.arctan(
34                 (2 / np.tan(b_rad)) *
35                 ((M**2 * np.sin(b_rad)**2 - 1) /
36                  (M**2 * (GAMMA + np.cos(2 * b_rad)) + 2))
37             )
38         )
39         if t >= theta_deg:
40             return b
41     return None
42
43 def downstream_mach(M, beta, theta):
44     b_rad = np.radians(beta)
45     Mn1 = M * np.sin(b_rad)
46     Mn2 = np.sqrt((1 + (GAMMA - 1) / 2 * Mn1**2) /
47                  (GAMMA * Mn1**2 - (GAMMA - 1) / 2))
48     return Mn2 / np.sin(np.radians(beta - theta))
49
50 fig, ax = plt.subplots(figsize=(10, 6))
51 theta0, p0 = shock_polar(M_inf, 0.0)
52 plt.plot(theta0, p0, '-', color='C0', linewidth=2, label=f'$M_{\infty}={M\_inf}$')
53
54 colors = ['C1', 'C2']
55 for i in range(len(deflections)):
56     beta = weak_shock_angle(M_inf, abs(deflections[i]))
57     if beta is None:
58         continue
59     M2 = downstream_mach(M_inf, beta, abs(deflections[i]))
60     if M2 >= 1.0:
61         b_rad = np.radians(beta)
62         p_ratio_at_shock = 1 + (2 * GAMMA / (GAMMA + 1)) * ((M_inf * np.sin(
63             b_rad))**2 - 1)
64         theta2, p2 = shock_polar(M2, theta_offset=deflections[i])

```

```

64     plt.plot(theta2, p_ratio_at_shock * p2, '--', linewidth=1.5, color=
        colors[i], label=rf'$\theta={deflections[i]:+d}^\circ$; M2={M2:.2f}$')
65
66 ax.set_xlabel(r'Deflection Angle $\theta$ (deg)')
67 ax.set_ylabel(r'Pressure Ratio ($P / P_\infty$)')
68 ax.set_xlim(-50, 50)
69 ax.set_ylim(bottom=0)
70 ax.legend(loc='lower right')
71 ax.minorticks_on()
72 ax.grid(True, which='major', linestyle=':', linewidth=1.5, alpha=0.8)
73 ax.grid(True, which='minor', linestyle=':', linewidth=1.0, alpha=0.4)
74 plt.tight_layout()
75 plt.savefig("shockPolar_typeI.png", dpi=300, bbox_inches='tight', transparent=
    True)
76 plt.show()

```

Listing 3: Scramjet intake oblique shock design (intake.py).

```

1  import math
2
3  GAMMA = 1.4
4  M1, PI, H_ISO = 6.0, 20.0, 0.05
5
6  def tbm(M, b):
7      sb = math.sin(b)
8      Mn1sq = (M * sb) ** 2
9      return math.atan(2*(Mn1sq-1) / (math.tan(b)*(M**2*(GAMMA+math.cos(2*b))+2)))
10
11 def solve_beta(M, theta_deg):
12     theta = math.radians(theta_deg)
13     mu = math.asin(1/M)
14     beta = mu + math.radians(3)
15     for _ in range(300):
16         f = tbm(M, beta) - theta
17         df = (tbm(M, beta+1e-7) - tbm(M, beta-1e-7)) / 2e-7
18         if abs(df) < 1e-15: break
19         beta = max(mu+1e-6, min(beta + max(min(-f/df, math.radians(0.5)), math.
            radians(-0.5)), math.pi/2-1e-6))
20     if abs(f) < 1e-10: break
21     return beta
22
23 def p_ratio(M, b):
24     Mn1 = M * math.sin(b)
25     return (2*GAMMA*Mn1**2 - (GAMMA-1)) / (GAMMA+1)
26
27 def rho_ratio(M, b):
28     Mn1 = M * math.sin(b)
29     return ((GAMMA+1)*Mn1**2) / ((GAMMA-1)*Mn1**2 + 2)
30
31 def M_down(M, b, theta):
32     Mn1 = M * math.sin(b)
33     Mn2sq = (1 + (GAMMA-1)/2*Mn1**2) / (GAMMA*Mn1**2 - (GAMMA-1)/2)

```

```

34     return math.sqrt(Mn2sq) / math.sin(b - theta)
35
36 def max_deflection(M):
37     mu = math.asin(1/M)
38     return max(tbm(M, math.radians(b)) for b in range(int(math.degrees(mu))+1,
39                                                         90))
40
41 def solve_theta(M1, pi_target):
42     lo, hi = 0.1, math.degrees(max_deflection(M1)) * 0.95
43     for _ in range(120):
44         theta = 0.5*(lo+hi)
45         b1 = solve_beta(M1, theta)
46         M2 = M_down(M1, b1, math.radians(theta))
47         if theta >= math.degrees(max_deflection(M2)):
48             hi = theta; continue
49         b2 = solve_beta(M2, theta)
50         p31 = p_ratio(M1, b1) * p_ratio(M2, b2)
51         if abs(p31 - pi_target) < 1e-8: break
52         lo, hi = (theta, hi) if p31 < pi_target else (lo, theta)
53     return 0.5*(lo+hi)
54
55 theta_deg = solve_theta(M1, PI)
56 theta     = math.radians(theta_deg)
57 b1        = solve_beta(M1, theta_deg)
58 M2        = M_down(M1, b1, theta)
59 b2        = solve_beta(M2, theta_deg)
60 M3        = M_down(M2, b2, theta)
61 p21       = p_ratio(M1, b1)
62 p32       = p_ratio(M2, b2)
63 p31       = p21 * p32
64
65 m         = math.tan(theta - b2)
66 x_s       = H_ISO * (m - math.tan(b1)) / (m * (math.tan(b1) - math.tan(theta)))
67 L_ramp    = x_s / math.cos(theta)
68
69 t1 = 1 + 0.5*(GAMMA-1)*M1**2
70 t3 = 1 + 0.5*(GAMMA-1)*M3**2
71 tpr = p31 * (t3/t1) ** (GAMMA/(GAMMA-1))
72
73 print(f"Inputs:  M={M1}  p3/p1={PI}  H_iso={H_ISO} m")
74 print(f"Theta (ramp angle):  {theta_deg:.4f} deg")
75 print(f"Beta1 (LE shock):     {math.degrees(b1):.4f} deg")
76 print(f"Beta2 (cowl shock):   {math.degrees(b2):.4f} deg")
77 print(f"Ramp length:         {L_ramp:.5f} m")
78 print(f"Total pressure recovery: {tpr:.5f}")

```